

Lógica de Programação

Prof.^a Kecia Aline Marques Ferreira

DECOM | Departamento de
Computação



Lógica de Programação

Ponteiros

Ponteiros

- Expressões com ponteiros
- Ponteiros e matrizes
- Indireção múltipla
- O valor **NULL**
- Cuidados com o uso de ponteiros

Expressões com Ponteiros

Atribuição

Atribuição: uma variável ponteiro pode ser atribuída a outra.

```
#include <stdio.h>
```

```
void main() {
```

```
    int x;
```

```
    int *p1, *p2;
```

```
    printf("Valor de p1 antes de inicializar: %p", p1);
```

```
    printf("Valor de p2 antes de inicializar: %p", p2);
```

```
    p1 = &x;
```

```
    p2 = p1;
```

```
    printf("Valores de p1 e p2 apos os comandos: %p - %p", p1, p2);
```

```
}
```

Saída do programa:

Valor de p1 antes de inicializar: 0000000000000010

Valor de p2 antes de inicializar: 00000000003513D0

Valores de p1 e p2 apos os comandos: 000000000023FE3C - 000000000023FE3C

Aritmética

Existem apenas duas operações aritméticas que podem ser realizadas com ponteiros: **adição e subtração**.

A adição de um inteiro **n** a um ponteiro **p** faz **p** apontar para a **n-ésima** posição adiante daquela para a qual **p** aponta, considerando a quantidade de bytes ocupados pelo tipo base do ponteiro.

A subtração de um inteiro **n** de um ponteiro **p** faz **p** apontar para a **n-ésima** posição anterior àquela para a qual **p** aponta, considerando a quantidade de bytes ocupados pelo tipo base do ponteiro.

Aritmética

Incremento: cada vez que o ponteiro é incrementado, ele aponta para a posição de memória do próximo elemento do seu tipo base.

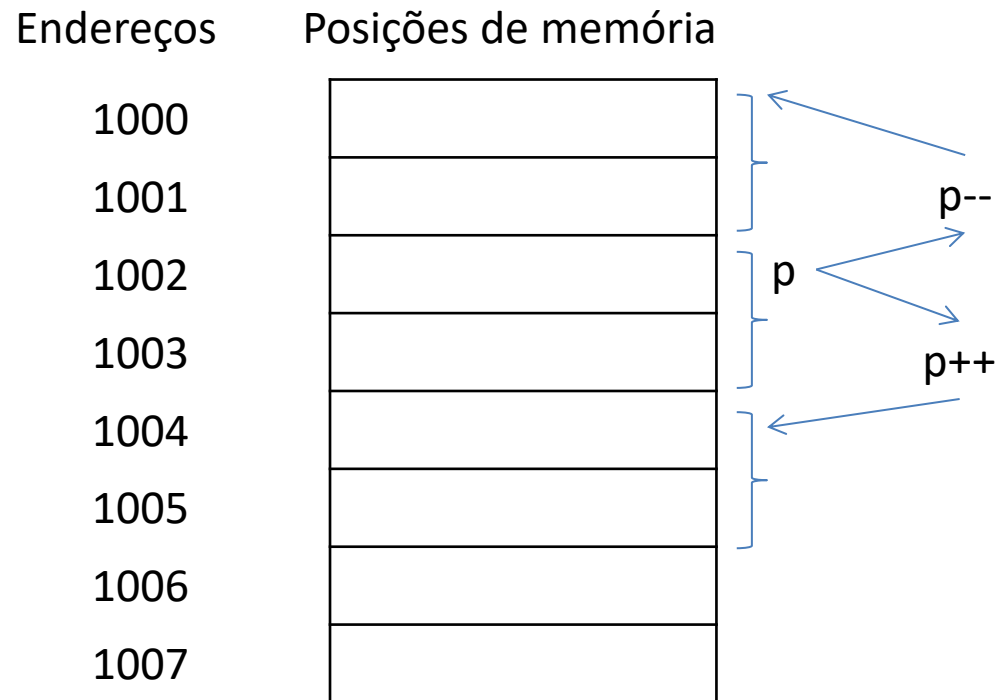
Decremento: cada vez que o ponteiro é decrementado, ele aponta para a posição de memória do elemento anterior do seu tipo base.

Aritmética

Considere um ponteiro p para inteiro e que um inteiro ocupa 2 bytes:

A operação $p++$ faz p apontar para a posição que está 2 bytes adiante.

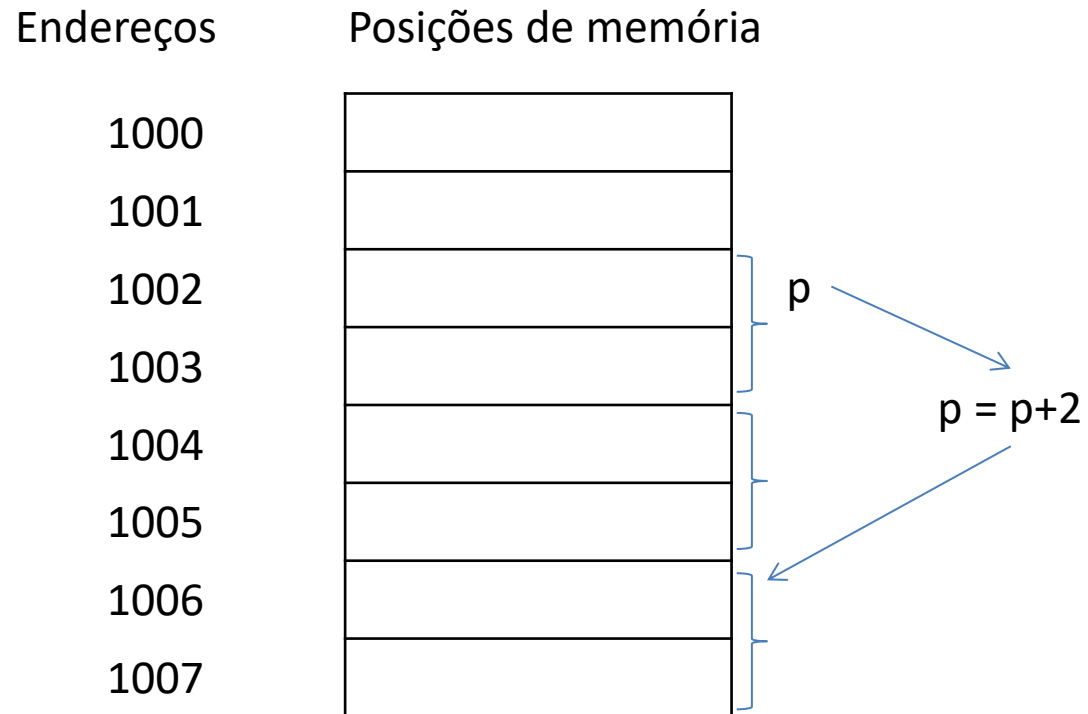
A operação $p--$ faz p apontar para a posição que está 2 bytes atrás.



Aritmética

Considere um ponteiro p para inteiro e que um inteiro ocupa 2 bytes

A operação $p = p + 2$ resulta conforme mostra a figura:



Comparação

É possível comparar o valor de dois ponteiros em uma operação relacional

Ex.: Sejam **p** e **q** ponteiros:

$p > q$: resulta em verdadeiro se o endereço da posição de memória apontada por p for maior do que a de q

$p == q$: resulta em verdadeiro se o endereço da posição de memória apontada por p é a mesma de q

Comparação

```
#include <stdio.h>

void main(){

    int x, y;
    int *p1, *p2;

    p1 = &x;
    p2 = &y;

    printf("\np1: %p", p1);
    printf("\np2: %p", p2);

    if (p1 > p2) printf("\np1 é maior do que p2");
    else printf("\np1 não é maior do que p2");
}
```

Saída do programa:

p1: 000000000023FE3C
p2: 000000000023FE38
p1 é maior do que p2

Ponteiros e Matrizes

Ponteiros e Matrizes

Uma matriz é um conjunto de dados do mesmo tipo (estrutura de dados heterogênea).

Quando uma matriz é definida, é reservada uma área de memória em caibam os seus dados.

Por exemplo, para um vetor de inteiros de tamanho 5, são reservadas 5 posições contíguas na memória, cada uma correspondente ao tamanho de um inteiro.

A primeira posição de memória reservada para a matriz é o endereço da matriz.

Dessa forma, pode-se manipular ponteiros para acessar as posições de uma matriz.

Ponteiros e Matrizes

```
int vet[5];
```

Endereço	Valor	Posição do vetor
1000		0
1001		
1002		1
1003		
1004		2
1005		
1006		3
1007		
1008		4
1009		

Ponteiros e Matrizes

Considere o vetor e o ponteiro definidos a seguir:

```
int vet[5], *p;  
p = vet;
```

Nesse trecho, p recebe o endereço da primeira posição do vetor.

Dessa forma, para acessar a quarta posição do vetor, as duas formas a seguir são possíveis:

```
vet[4]    ou    *(p+4)
```

Ponteiros e Matrizes

```
#include <stdio.h>

void main(){
    int vet[5] = {23,45,6,78,12}, *p;

    p = vet;

    printf("%d", *p);
    printf("\n%d - %d", vet[4], *(p+4));
}
```

Saída do programa:

23
12 - 12

Exemplo

```
include <stdio.h>
#include <string.h>

void main(){
    char str[50] = "Engenharia de Computacao";

    //Imprimindo o vetor utilizando-se indexação
    for(int i=0; i<strlen(str); i++)
        putchar(str[i]);

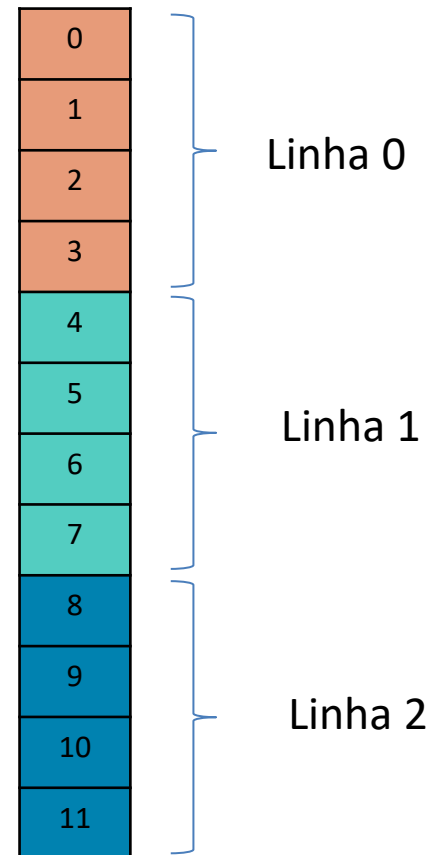
    //Imprimindo o vetor utilizando-se ponteiro
    char *p = str;

    while(*p) putchar(*p++);
}
```

Ponteiros e Matrizes

As posições de uma matriz bidimensional são armazenados de forma contígua na memória.

0	1	2	3
4	5	6	7
8	9	10	11



Exemplo

```
#include <stdio.h>

void main(){
    int mat[3][4] = {{0,1,2,3},{4,5,6,7},{8,9,10,11}};

    int *p;
    p = mat;

    for (int i=0; i<12; i++, p++)
        printf("%d - ", *p);
}
```

Saída do programa:

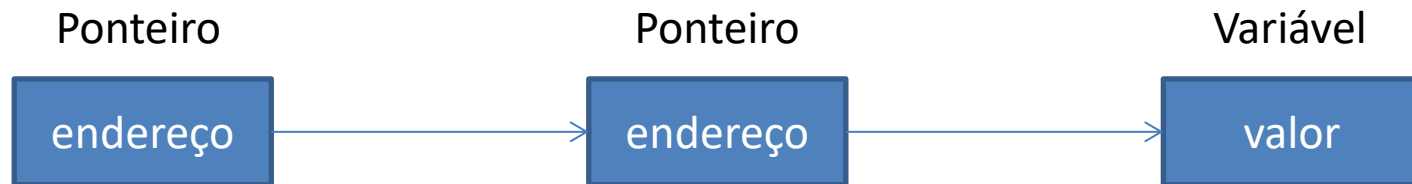
0 - 1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9 - 10 - 11 -

Indireção Múltipla

Indireção Múltipla

É possível que uma variável ponteiro aponte para outra variável ponteiro.

Essa situação é chamada de **indireção múltipla** ou **ponteiro para ponteiro**.



Exemplo

```
#include <stdio.h>

void main(){
    int x, *p, **q;

    x= 25;
    p = &x;
    q = &p;

    printf("\n%p - %p", p, q);
    printf("\n%d", x);
    printf("\n%d", *p);
    printf("\n%p", *q);
    printf("\n%d", **q);
}
```

Saída do programa:

```
000000000023FE44 - 000000000023FE38
25
25
000000000023FE44
25
```

O valor **NULL**

O valor **NULL**

O valor **NULL** é utilizado para indicar que o ponteiro não aponta para algum lugar da memória.

```
p = NULL;
```

Quando um ponteiro é declarado e não é inicializado com algum valor, ele contém um endereço desconhecido (lixo) e que aponta para qualquer área da memória.

Se o ponteiro for usado no programa, ele poderá causar um erro não só no programa, mas em qualquer outro programa que esteja em execução.

O valor NULL

```
#include <stdio.h>

void main(){
    int *p;

    printf("\n%p", p);

    p = NULL;

    printf("\n%p", p);
}
```

Saída do programa:

```
000000000000000010
000000000000000000
```

Cuidado! Área de memória desconhecida!

Cuidados com o Uso de Ponteiros

Cuidados com o Uso de Ponteiros

“Nada pode trazer mais problemas do que um ponteiro selvagem!”

Herbert Schildt

Ponteiros são uma ferramenta poderosa, pois permitem ao programador a manipulação de dados em baixo nível.

Porém, o uso de ponteiros requer muito cuidado!

Quando um ponteiro recebe um valor incorreto, isso pode:

- gerar erro não só no próprio programa, mas em outros programas em execução, inclusive no sistema operacional;
- introduzir falhas de segurança nos programas (porque pode acessar áreas de memória que não são a do próprio programa);
- ser um erro muito difícil de identificar.

Cuidados com o Uso de Ponteiros

São fontes de erro comuns em programas que utilizam ponteiros:

- Não inicializar o ponteiro.
- Ao incrementar/decrementar o valor de um ponteiro, não garantir que o ponteiro não acessará posições além dos limites (do vetor, por exemplo).